

Compte-rendu

Devoir N 1 : kotlin

Prénom : Mohamed

Nom : EL HAIRCHE

Groupe : DEVOAM202

Sommaire :

Introduction	3
Exercice 1 : Système de gestion de bibliothèque	4
Classe Personne	4
Classe Utilisateur	5
Classe Livre	6
Classe Emprunt	8
Classe GestionBibliotheque	10
Classe Bibliotheque	11
fun main : Initialisation et appels	11
Affichage du système de bibliothèque	12
Exercice 2 : Système de gestion d'un parc automobile	13
Partie 1 : Classes de base	13
Vehicule	13
Voiture	15
Moto	16
Partie 2 : Gestion des réservations	17
Conducteur	17
Reservation	18
Partie 3 : Gestion du parc	19
ParcAutomobile	19
Fonctions de gestion et d'affichage	20
Partie 4 : Gestion des exceptions	21
VehiculeIndisponibleException	21
VehiculeNonTrouveException	21
fun main et réservation	22
Affichage du système	23
Conclusion	24

Introduction :

Dans le cadre du module Kotlin (M202), ce devoir a pour objectif de mettre en pratique les concepts de la programmation orientée objet (POO) en Kotlin.

Le travail est composé de deux exercices : la création d'un système de gestion de bibliothèque et la création d'un système de gestion d'un parc automobile. Ces exercices permettent d'utiliser l'héritage, l'encapsulation, le polymorphisme, les classes abstraites et la gestion des exceptions.

Exercice 1 : cr  ation d'un syst  me de gestion de biblioth  que.

Classe Personne :

```
1  open class Personne(  
2      val nom: String,  
3      val prenom: String,  
4      val email: String  
5  ) {  
6  
7      open fun afficherInfos() {  
8          println("Nom : $nom")  
9          println("Pr  nom : $prenom")  
10         println("Email : $email")  
11     }  
12 }
```

Figure1 :creation de la classe Personne

Classe "Personne" :

Cette classe contient les informations g  n  rales d'une personne : le nom, le pr  nom et l'adresse email. Elle contient   galement la m  thode "afficherInfos()" pour afficher ces informations.

```
open fun afficherInfos() {  
    println("Nom : $nom")  
    println("Pr  nom : $prenom")  
    println("Email : $email")  
}
```

Figure2 :creation de la m  thode afficherInfos

Cette m  thode permet d'afficher les informations principales d'une personne, notamment son nom, son pr  nom et son adresse e-mail.

Ann  e scolaire : 2025/2026

Classe Utilisateur :

```
class Utilisateur( 3 Usages
    nom: String,
    prenom: String,
    email: String,
    val idUtilisateur: Int
) : Personne(nom, prenom, email) {
```

Figure3 :creation de la classe Utilisateur

La classe Utilisateur représente une personne qui utilise les services de la bibliothèque. Elle hérite de la classe Personne, donc elle récupère ses propriétés et sa méthode.

```
fun emprunterLivres(livre: Livre, dateEmprunt: String) {
    if (livre.disponiblePourEmprunt()) {
        livre.mettreAJourStock( nouveauStock = livre.nombreExemplaires - 1)
        val emprunt = Emprunt(
            utilisateur = this,
            livre,
            dateEmprunt,
            dateRetour = null
        )
        emprunts.add(emprunt)
        println("$prenom $nom a emprunté le livre : ${livre.titre}")
    } else {
        println("Le livre ${livre.titre} n'est pas disponible.")
    }
}
```

Figure4 :creation de la méthode fun emprunterLivres

Cette méthode permet à un utilisateur d'emprunter un livre. Elle vérifie d'abord si le livre est disponible. Si un exemplaire existe, le nombre d'exemplaires est diminué et un nouvel objet Emprunt est créé puis ajouté à la liste des emprunts de l'utilisateur.

```
fun afficherEmprunts() {  
    println("Emprunts de $prenom $nom :")  
  
    if (emprunts.isEmpty()) {  
        println("Aucun emprunt.")  
    } else {  
        for (emprunt in emprunts) {  
            emprunt.afficherDetails()  
        }  
    }  
}
```

Figure5 :creation de la méthode afficherEmprunts

Cette méthode permet d'afficher tous les emprunts appartenant à l'utilisateur. Elle parcourt la liste emprunts et affiche les informations de chaque emprunt.

Classe Livre :

```
13  
14 class Livre(  
15     val titre: String,  
16     val auteur: String,  
17     val isbn: String,  
18     nombreExemplaires: Int  
19 ) {  
20     var nombreExemplaires: Int = nombreExemplaires 3 Usages  
21 }
```

Figure6 :creation de la classe Livre

Nous avons créé la classe Livre avec son titre, son auteur, son ISBN et le nombre d'exemplaires.

```
fun afficherDetails() { 1 Usage
    println("Titre : $titre")
    println("Auteur : $auteur")
    println("ISBN : $isbn")
    println("Nombre d'exemplaires : $nombreExemplaires")
}
fun disponiblePourEmprunt(): Boolean { 1 Usage
    return nombreExemplaires > 0
}
```

Figure7 :creation de les deux methodes afficherDetails methode disponiblePourEmprunt

Cette méthode affiche toutes les informations concernant le livre : son titre, son auteur, son ISBN et le nombre d'exemplaires disponibles, et le methode disponiblePourEmprunt cette méthode vérifie si le livre peut être emprunté.

```
fun mettreAJourStock(nouveauStock: Int) { 2 Usages
    nombreExemplaires = nouveauStock
}
```

Figure8 :creation de methode mettreAJourStock

Cette méthode permet de modifier le nombre d'exemplaires disponibles. Elle est utilisée notamment après un emprunt ou après le retour d'un livre.

Classe Emprunt :

```
class Emprunt( 2 Usages
    val utilisateur: Utilisateur,
    val livre: Livre,
    val dateEmprunt: String,
    var dateRetour: String
) {
```

Figure9 :creation de la classe Emprunt

La classe Emprunt représente l'opération d'emprunt d'un livre par un utilisateur.

Elle contient :

- utilisateur : l'utilisateur qui emprunte le livre.
- livre : le livre emprunté.
- dateEmprunt : date à laquelle le livre a été emprunté.
- dateRetour : date de retour du livre.

```
fun afficherDetails() { 1 Usage
    println("Utilisateur : ${utilisateur.prenom} ${utilisateur.nom}")
    println("Livre : ${livre.titre}")
    println("Date d'emprunt : $dateEmprunt")
    println("Date de retour : ${dateRetour} ?: "Non retourné")
}
```

Figure10 :creation de la methode afficherDetails

Cette méthode affiche les informations de l'emprunt : l'utilisateur, le livre, la date d'emprunt et éventuellement la date de retour.

```
fun retournerLivre() {
    if (dateRetour == null) {
        dateRetour = "03/10/2026"
        livre.mettreAJourStock( nouveauStock = livre.nombreExemplaires + 1)
        println("Le livre ${livre.titre} a été retourné.")
    } else {
        println("Ce livre a déjà été retourné.")
    }
}
```

Figure11 :creation de la methode retournerLivre

Cette méthode permet de terminer un emprunt. Elle renseigne la date de retour et augmente le nombre d'exemplaires disponibles du livre.

Classe GestionBibliotheque :

```

abstract class GestionBibliotheque { 1 Usage 1 Implementation

    val utilisateurs = mutableListOf<Utilisateur>() 1 Usage
    val livres = mutableListOf<Livre>() 3 Usages

    fun ajouterUtilisateur(utilisateur: Utilisateur) {
        utilisateurs.add(utilisateur)
    }
  
```

Figure12 :creation de la class GestionBibliotheque et la methode ajouterUtilisateur

GestionBibliotheque est une **classe abstraite**. Elle sert de base pour gérer les éléments principaux de la bibliothèque, et la methode cette méthode ajoute un utilisateur dans la liste des utilisateurs de la bibliothèque.

```

    fun ajouterLivre(livre: Livre) {
        livres.add(livre)
    }

    fun afficherTousLesLivres() {
        println("tout les livres " )

        for (livre in livres) {
            livre.afficherDetails()
            println("tout les livres")
        }
    }
  
```

Figure13 :creation de les deux methodes ajouterLivre methode afficherTousLesLivres

La methodes ajouterLivre cette méthode ajoute un livre dans la liste des livres, et le methode afficherTousLesLivres cette méthode parcourt la liste des livres et affiche les informations de chaquelivre. La classe est abstraite, donc elle **ne peut pas être** instanciée directement.

Classe Bibliotheque :

```
class Bibliotheque : GestionBibliotheque() {  
  
    fun rechercherLivreParTitre(titre: String): Livre? {  
        return livres.find {  
            it.titre.equals( other = titre, ignoreCase = true)  
        }  
    }  
}
```

Figure14 :creation de la classe Bibliotheque et la methodes rechercherLivreParTitre

La classe Bibliotheque hérite de GestionBibliotheque et représente l'implémentation concrète du système de bibliothèque. Nous avons ajouté la méthode rechercherLivreParTitre() afin de pouvoir retrouver un livre dans la liste à partir de son titre.

Fun main :

```
val livre1 = Livre(  
    titre = "Le Petit Prince",  
    auteur = "Antoine de Saint-Exupéry",  
    isbn = "978-2070612758",  
    nombreExemplaires = 3  
)  
  
val utilisateur1 = Utilisateur(  
    nom = "Hairche",  
    prenom = "Mohamed",  
    email = "mohamed@gmail.com",  
    idUtilisateur = 1  
)
```

Figure15 : Intialisation des variable

Dans cette première partie, nous avons créé un objet Livre contenant le titre, l'auteur, l'ISBN et le nombre d'exemplaires. Nous avons également créé un objet Utilisateur contenant son nom, son prénom, son e-mail et son identifiant.

```

val bibliotheque = Bibliotheque()

bibliotheque.ajouterLivres(livre1)
bibliotheque.ajouterUtilisateur(utilisateur1)

utilisateur1.emprunterLivres(livre1, dateEmprunt = "01/10/2026")
bibliotheque.afficherTousLesLivres()
utilisateur1.afficherInfos()
utilisateur1.afficherEmprunts()
val emprunt = utilisateur1.emprunts[0]
emprunt.retournerLivres()

livre1.afficherDetails()
emprunt.afficherDetails()
}
  
```

Figure16 : Appel de fonction

Nous avons d'abord créé une bibliothèque et ajouté le livre et l'utilisateur à l'aide des méthodes `ajouterLivres()` et `ajouterUtilisateur()`. Ensuite, nous avons permis à l'utilisateur d'emprunter le livre avec `emprunterLivres()`, ce qui met à jour le stock et ajoute l'emprunt à sa liste. Nous avons ensuite affiché les informations du livre, de l'utilisateur et de l'emprunt. Enfin, nous avons récupéré l'emprunt et utilisé `retournerLivres()` pour enregistrer le retour du livre et mettre à jour le nombre d'exemplaires disponibles.

```

tout les livres
Titre : Le Petit Prince
Auteur : Antoine de Saint-Exupéry
ISBN : 978-2070612758
Nombre d'exemplaires : 2
tout les livres
Nom : Hairche
Prénom : Mohamed
Email : mohamed@gmail.com
Emprunts de Mohamed Hairche :
Utilisateur : Mohamed Hairche
Livre : Le Petit Prince
Date d'emprunt : 01/10/2026
Date de retour : Non retourné
  
```

Figure17 : Affichage de système de gestion de bibliothèque

Le programme permet de gérer un emprunt et le retour d'un livre. L'utilisateur emprunte le livre, puis les informations du livre, de l'utilisateur et de l'emprunt sont affichées. Lors du retour, la date de retour est enregistrée et le nombre d'exemplaires disponibles est mis à jour automatiquement.

Exercice 2 :

Partie 1 : Classes de Base

Classe Vehicule :

```
abstract class Vehicule( 2 Usages 2 Implementations
    val immatriculation: String,
    val marque: String,
    val modele: String,
    var kilometrage: Int,
    var disponible: Boolean = true
) {

    open fun afficherDetails() { 2 Overrides
        println("Immatriculation : $immatriculation")
        println("Marque : $marque")
        println("Modèle : $modele")
        println("Kilométrage : $kilometrage km")
        println("Disponible : $disponible")
    }
}
```

Figure1 : Creation de la classe Vehicule

La classe Vehicule est une classe abstraite qui contient les informations communes à tous les véhicules : immatriculation, marque, modele, kilometrage et disponible. est le fonction afficherDetails qui affiche les informations du véhicule.

```
fun estDisponible(): Boolean {  
    return disponible  
}  
fun marquerIndisponible() {  
    disponible = false  
}
```

Figure2 : Creation de les methodes estDisponible et marquerIndisponible

La fonction estDisponible qui v  rifie si le v  hicule est disponible et retourne true ou false. et la fonction marquerIndisponible() : rend le v  hicule indisponible lors d'une r  servation.

```
fun marquerDisponible() {  
    disponible = true  
}  
fun mettreAJourKilometrage(km: Int) {  
    kilometrage = km  
}
```

Figure 3 : Creation de les methodes marquerDisponible et mettreAJourKilometrage

La fonction marquerDisponible qui rend le v  hicule disponible apr  s la fin d'une r  servation. et la fonction mettreAJourKilometrage qui modifie le kilom  trage du v  hicule.

Class Voiture :

```
class Voiture(  
    immatriculation: String,  
    marque: String,  
    modele: String,  
    kilometrage: Int,  
    val nombrePortes: Int,  
    val typeCarburant: String  
) : Vehicule(  
    immatriculation,  
    marque,  
    modele,  
    kilometrage  
) {
```

Figure 4 : Creation de la classe Voiture

La classe Voiture hérite de Vehicule et ajoute deux propriétés : nombrePortes et typeCarburant.

```
override fun afficherDetails() {  
    println("Voiture")  
    println("Immatriculation : $immatriculation")  
    println("Marque : $marque")  
    println("Modèle : $modele")  
    println("Kilométrage : $kilometrage km")  
    println("Nombre de portes : $nombrePortes")  
    println("Type de carburant : $typeCarburant")  
    println("Disponible : $disponible")  
}  
}
```

Figure 5 : Creation de methode afficherDetails

La fonction afficherDetails qui redéfinit la méthode de la classe mère pour afficher les informations spécifiques d'une voiture.

Classe Moto :

```
class Moto(  
    immatriculation: String,  
    marque: String,  
    modele: String,  
    kilometrage: Int,  
    val cylindree: Int  
) : Vehicule(  
    immatriculation,  
    marque,  
    modele,  
    kilometrage  
) {
```

Figure 6: Creation de methode afficherDetails

La classe Moto hérite également de Vehicule. Elle ajoute la propriété cylindree, exprimée en cm³.

```
override fun afficherDetails() {  
    println("moto")  
    println("Immatriculation : $immatriculation")  
    println("Marque : $marque")  
    println("Modèle : $modele")  
    println("Kilométrage : $kilometrage km")  
    println("Cylindrée : $cylindree cm³")  
    println("Disponible : $disponible")  
}
```

Figure 7: Creation de methode afficherDetails

La fonction afficherDetails qui affiche les informations spécifiques à la moto.

Partie 2 : Gestion des Réservations

Classe Conducteur :

```
class Conducteur( 1 Usage
    val nom: String,
    val prenom: String,
    val numeroPermis: String
) {

    fun afficherDetails() {
        println("Nom : $nom")
        println("Prénom : $prenom")
        println("Numéro de permis : $numeroPermis")
    }
}
```

Figure 8: Creation de la classe Conducteur et la methode afficherDetails

La classe Conducteur représente la personne qui peut effectuer une réservation. Elle contient trois informations : le nom, le prénom et le numéro du permis.

La méthode afficherDetails() permet d'afficher les informations du conducteur.

Classe Reservation :

```

class Reservation( 2 Usages
    val vehicule: Vehicule,
    val conducteur: Conducteur,
    val dateDebut: String,
    val dateFin: String,
    val kilometrageDebut: Int,
    var kilometrageFin: Int?
) { fun cloturerReservation(kilometrageRetour: Int) {
    kilometrageFin = kilometrageRetour
    vehicule.mettreAJourKilometrage( km = kilometrageRetour)
    vehicule.marquerDisponible()
    println("Réservation clôturée.")
  }
}
  
```

Figure 9: Creation de la classe reservation et la methode cloturerReservation

Reservation permet de représenter une réservation entre un **conducteur** et un **véhicule**. Elle contient le véhicule réservé, le conducteur, la date de début, la date de fin et les kilométrages et kilometrageFin est nullable (Int?) car au moment où la réservation commence, le kilométrage de retour n'est pas encore connu , et la fonction cloturerReservation(kilometrageRetour) qui termine la réservation, enregistre le kilométrage de retour et rend le véhicule disponible.

```

fun afficherDetails() { 1 Usage
    println("Reservation ")
    println("Véhicule : ${vehicule.immatriculation}")
    println(
        "Conducteur : ${conducteur.prenom} ${conducteur.nom}"
    )
    println("Date début : $dateDebut")
    println("Date fin : $dateFin")
    println("Kilométrage début : $kilometrageDebut")
    println(
        "Kilométrage fin : ${kilometrageFin} ?: "Non renseigné"
    )
}
}
  
```

Figure 10: Creation de methode afficherDetails

La fonction afficherDetails qui affiche toutes les informations concernant la réservation.

Partie 3 : Gestion du Parc :

Class ParcAutomobile

```

class ParcAutomobile {
    val vehicules = mutableListOf<Vehicule>() 4 Usages
    val reservations = mutableListOf<Reservation>() 3 Usages
    fun ajouterVehicule(vehicule: Vehicule) {
        vehicules.add(vehicule)
    }
}
  
```

Figure 11: Creation de la class ParcAutomobile et le methode ajouterVehicule

Cette classe permet de gérer les véhicules du parc ainsi que les réservations. Elle contient deux listes : une pour les véhicules et une autre pour les réservations , et aussi la fonction ajouterVehicule cette fonction permet d'ajouter un nouveau véhicule dans la liste des véhicules du parc.

```

fun supprimerVehicule(immatriculation: String) { vehicules.removeIf {
    it.immatriculation == immatriculation } }
fun reserverVehicule(
    immatriculation: String,
    conducteur: Conducteur,
    dateDebut: String,
    dateFin: String) {
    val vehicule = vehicules.find {
        it.immatriculation == immatriculation
    } ?: throw VehiculeNonTrouveException(
        "Le véhicule avec l'immatriculation $immatriculation n'a pas été trouvé.")
    if (!vehicule.estDisponible()) {
        throw VehiculeIndisponibleException(
            "Le véhicule $immatriculation n'est pas disponible.") }
    val reservation = Reservation(
        vehicule, conducteur, dateDebut, dateFin,
        kilometrageDebut = vehicule.kilometrage,
        kilometrageFin = null)
    reservations.add(reservation)
    vehicule.marquerIndisponible()
    println(
        "Réservation effectuée pour le véhicule $immatriculation."
    ) }
}
  
```

Figure 12: Creation de la class ParcAutomobile et le methode ajouterVehicule

La fonction supprimerVehicule cette fonction permet de supprimer un véhicule du parc en utilisant son numéro d'immatriculation et la fonction reserverVehicule qui

Cette fonction permet de réserver un véhicule pour un conducteur. Elle vérifie que le véhicule existe et qu'il est disponible avant de créer la réservation.

```
fun afficherVehiculesDisponibles() {  
    println("VÉHICULES DISPONIBLES")  
    for (vehicule in vehicules) {  
        if (vehicule.estDisponible()) {  
            vehicule.afficherDetails()  
            println("")  
        } }  
}  
fun afficherReservations() {  
    println("RÉSERVATIONS")  
    if (reservations.isEmpty()) {  
        println("Aucune réservation.")  
    } else {  
        for (reservation in reservations) {  
            reservation.afficherDetails()  
            println("")  
        }  
    }  
}
```

Figure 13: Creation de la methode afficherVehiculesDisponibles afficherReservations

La fonction `afficherVehiculesDisponibles` Cette fonction permet d'afficher uniquement les véhicules qui sont actuellement disponibles pour une réservation et la fonction `afficherReservations` Cette fonction permet d'afficher les informations de toutes les réservations enregistrées dans le parc.

Partie 4 : Gestion des Exceptions :

Classe VehiculeIndisponibleException et VehiculeNonTrouveException :

```

class VehiculeIndisponibleException(message: String) : Exception(message) 1 Usage
class VehiculeNonTrouveException(message: String) : Exception(message) 1 Usage
    
```

Figure 14: Creation de la class afficher VehiculeIndisponibleExceptio VehiculeNonTrouveException

La classe VehiculeIndisponibleException cette classe représente une exception personnalisée utilisée lorsqu'on essaie de réserver un véhicule qui est déjà indisponible et la classe cette classe représente une exception personnalisée utilisée lorsqu'un véhicule recherché n'existe pas dans le parc.

Fun main :

```

fun main() {

    val voiture1 = Voiture(
        immatriculation = "12345-A-1", marque = "Dacia", modele = "Logan", kilometrage = 50000,
        nombrePortes = 4, typeCarburant = "Essence")
    val moto1 = Moto(
        immatriculation = "67890-B-2",
        marque = "Yamaha", modele = "MT-07", kilometrage = 20000, cylindree = 689)
    val conducteur1 = Conducteur(
        nom = "Hairche", prenom = "Mohamed", numeroPermis = "P123456")
    val parc1 = ParcAutomobile()
}
    
```

Figure15 : Intialisation des variable

Dans cette partie, nous avons créé les différents objets nécessaires au fonctionnement du programme :

- voiture1 : représente une voiture avec son immatriculation, sa marque, son modèle, son kilométrage, son nombre de portes et son type de carburant.
- moto1 : représente une moto avec son immatriculation, sa marque, son modèle, son kilométrage et sa cylindrée.

- conducteur1 : représente le conducteur avec son nom, son prénom et son numéro de permis.
- parc1 : représente le parc automobile qui va contenir les véhicules et gérer les réservations.

```
parc1.ajouterVehicule(voiture1)
parc1.ajouterVehicule(moto1)
println("Véhicules disponibles :")
parc1.afficherVehiculesDisponibles()
println("Conducteur :")
conducteur1.afficherDetails()
parc1.reserverVehicule(
    voiture1.immatriculation,
    conducteur1,
    dateDebut = "01/10/2026", dateFin = "05/10/2026")
println("Après la réservation :")
parc1.afficherVehiculesDisponibles()
println("Réservations :")
parc1.afficherReservations()
```

Figure16 : Appel de fonction

Dans cette partie, nous avons d'abord ajouté voiture1 et moto1 au parc1 avec ajouterVehicule().

Ensuite, afficherVehiculesDisponibles() permet d'afficher les véhicules qui sont disponibles.

La fonction afficherDetails() affiche les informations du conducteur.

Après cela, reserverVehicule() permet de réserver voiture1 pour conducteur1 entre le 01/10/2026 et le 05/10/2026.

Enfin, nous affichons de nouveau les véhicules disponibles et les réservations enregistrées afin de vérifier que la réservation a bien été effectuée.

```
Véhicules disponibles :  
VÉHICULES DISPONIBLES  
Voitur  
Immatriculation : 12345-A-1  
Marque : Dacia  
Modèle : Logan  
Kilométrage : 50000 km  
Nombre de portes : 4  
Type de carburant : Essence  
Disponible : true  
  
moto  
Immatriculation : 67890-B-2  
Marque : Yamaha  
Modèle : MT-07  
Kilométrage : 20000 km  
Cylindrée : 689 cm3  
Disponible : true
```

Figure17/: Affichage de système de gestion d'un parc

Cette partie affiche les informations de la voiture et de la moto ainsi que leur état. La valeur true indique que les deux véhicules sont disponibles.

Conclusion :

Ce travail nous a permis de mettre en pratique les principaux concepts de la programmation orientée objet avec Kotlin. Nous avons utilisé les classes, l'héritage, les classes abstraites et le polymorphisme pour créer un système de gestion d'une bibliothèque et d'un parc automobile. Nous avons également appris à utiliser les fonctions pour gérer les différentes opérations, comme les emprunts, les réservations et la mise à jour de la disponibilité des objets. Enfin, la gestion des exceptions nous a permis de traiter correctement certaines erreurs et de rendre le programme plus organisé et fiable.